

# Parallel Implementation of Sub-quadratic time Approximate Counting of 0-1 Knapsack Problem

Uddeshya Jha<sup>1</sup>, Dharmveer Singh Sisodiya<sup>2</sup>, \*N Gopalakrishna Kini<sup>3</sup>[0000-0002-9293-7460]  
and Vivekananda Bhat K<sup>4</sup>[0000-0003-3411-9221]

<sup>1</sup>School of Computer Engineering, Manipal Institute of Technology  
Manipal Academy of Higher Education, Manipal, India  
mr.uddeshya@gmail.com

<sup>2</sup>School of Computer Engineering, Manipal Institute of Technology  
Manipal Academy of Higher Education, Manipal, India  
0.dharmveersingh@gmail.com

<sup>3</sup>School of Computer Engineering, Manipal Institute of Technology  
Manipal Academy of Higher Education, Manipal, India  
\*ng.kini@manipal.edu

<sup>4</sup>School of Computer Engineering, Manipal Institute of Technology  
Manipal Academy of Higher Education, Manipal, India  
kv.bhat@manipal.edu

**Abstract.** The 0-1 Knapsack problem serves as a widely studied example of NP-hard problems, arising in optimality, scheduling, and resource allocation. While traditional algorithms involving dynamic programming provide an exact solution, they have a pseudo-polynomial time complexity of  $O(nC)$ , which limits the practicality of the algorithm for larger instances. More recently, there is a theoretical advancement showing that the group dynamic programming with convolution via FFT can approximate the optimal 0-1 Knapsack model in sub-quadratic time. This paper will discuss the analysis and implementation of the divide/conquer FFT-based approximation model, specifically: (i) a sequential implementation as a baseline; (ii) an MPI implementation for parallelization and (iii) a CUDA based GPU-accelerated implementation utilizing the cuFFT libraries to achieve the maximum degree of parallelism when performing convolutions. Each implementation employs polynomial scaling/limitations, trimming in considerations of resolution, and a sparse representation to minimize computational "overhead," which produces closely approximated results to the original NP-hard Knapsack model. Empirical study shows that there are significant speedups with respect to the sequential baseline due to using MPI, while the CUDA implementation shows promise for even greater speedup based on multi-stream batched FFTs.

**Keywords:** Knapsack Problem, Parallel Computing, MPI, CUDA, GPU, Approximate Counting, Divide-and-Conquer, Fast Fourier Transformation, High-Performance Computing (HPC), Sub-quadratic.

## 1 Introduction

The 0-1 Knapsack Problem is an NP-hard optimization problem that has become widely studied because of the number of applications it has in numerous areas including scheduling, selecting financial portfolios, logistics, resource allocation, and much more. The Knapsack Problem is defined as the need to determine a sub-selection of items (with attribute weights and profits) that will produce maximum overall profit, while limiting total weight to a predefined maximum or capacity. Traditional techniques used to solve the Knapsack Problem are based on Bellman’s dynamic programming formulation [2], where the solution is guaranteed to be exact. However, these techniques have a pseudo-polynomial run-time complexity of  $O(nC)$  which can be prohibitively expensive to use for larger datasets, especially when the capacity of the knapsack is large.

Over the years, researchers have worked to design approximation schemes for better computational performance and consideration. Kellerer, Pferschy, and Pisinger [3] present a survey and extensive discussion of knapsack variants, heuristics, and dynamic programming approaches. Other works have had a similar focus on Fully Polynomial-Time Approximation Schemes (FPTAS), and similar research has focused on techniques to make the traditional dynamic programming more efficient through use of Fast Fourier Transform (FFT) techniques and by using min-plus convolution in [4][5][6]. The incorporation of FFT into running time for dynamic programming reduces compute time of convolution by a large factor: from  $O(n^2)$  to  $O(n \log(n))$  and further improving practical performance for large input sizes.

Feng and Jin [1] offered a major theoretical advancement in their divide-and-conquer grouped dynamic programming paradigm of FFT-based convolution mechanisms for approximate counting and optimization in sub-quadratic time  $O(n^{1.5})$ . Their framework scales items, trims polynomials, and writes under Turing’s model for sparsity with the objective of controlling the growth of computational time and error bounded rates. Nonetheless, this is primarily theoretical with little evidence is offered empirically in settings that are heterogeneous and/or in parallel.

The present work provides an empirical model by implementing the above framework and testing three computational execution models: (i) A sequential CPU-based implementation serving as the baseline, (ii) An MPI-based distributed parallel model emphasizing scalability across multiple processors, (iii) A GPU-accelerated CUDA implementation employing cuFFT for high-throughput polynomial merging. (All three implementations mentioned above use scaling, trimming, and grouped FFT-based convolutions as measure for valid empirical performances and theoretical sub-quadratic time performances.)

The results offer practical examples for the utility of the FFT knapsack algorithms under the real system constraints. We study performance as it scales with problem size, approximation parameter ( $\epsilon$ ), systems architecture, and discuss tradeoffs between accuracy, execution time, and resource utilization. This study contributes toward bridging the gap between theoretical algorithm design and practical high-performance realization and provides insights for hybrid CPU–GPU optimization in future frameworks.

## 2 Literature Survey

The 0–1 knapsack problem is a central model in combinatorial optimization, with a long history stretching from Bellman’s original dynamic programming formulation to contemporary fine-grained algorithmic research. Bellman’s pseudo-polynomial dynamic programming approach yields an exact algorithm running in  $O(nC)$  time for capacity  $C$ , which underlies much subsequent research into practical implementations and approximation schemes [2]. A major strand of work is concentrated on designing Fully Polynomial Time Approximation Schemes (FPTAS) and on structural reductions that reduce the dependence on  $n$  and  $C$  while controlling approximation error; surveys and monographs synthesize these developments and provide the canonical engineering and theoretical baseline for knapsack work [3].

A second major thread is the use of convolution as a core computational primitive for merging subset-sum or polynomial representations of item groups. In many divide-and-conquer and grouping strategies, each group is represented by a polynomial (or vector) whose coefficients encode achievable weights or counts, and the combination of groups reduces to convolution. Naive convolution is quadratic in the support size, but the application of Fast Fourier Transform (FFT) reduces this to near-linear (per convolution) work and thereby enables faster merging in practice. Bateni et al. demonstrated how combining FFT-based convolution with prediction and pruning heuristics leads to practical speedups in DP-based merges for knapsack and related problems [4].

On the theoretical front, recent advances have tackled the quadratic barrier more directly. Bringmann & Cassis (and follow-up works by Bringmann et al., Bringmann–Dürr–Polak, and others) developed frameworks that exploit bounded monotonicity, near-convexity, or rectangular balancing to accelerate convolution operations; these works achieve provable sub-quadratic running times for knapsack-like tasks in certain regimes and thereby provide a roadmap for asymptotic improvements beyond FFT-based heuristics [5][6][7][10]. In addition to the pure Fast Fourier Transform, other methods (e.g., monotone min-plus convolution, balancing and blocking) have developed in addition to detailing the structure of the instances for runtime assurances.

Variants of the Knapsack Problem such as Approximate Counting and Sampling add additional levels of technical detail. The works of Dyer, Gopalan et al. have laid early and fundamental groundwork in this area on dynamic programming by creating approximate-counting frameworks, showing that counting has distinct differences to decision or optimization and therefore requires greater levels of control when managing errors, and use of either random rounding or sampling will enable a practical method to achieve approximate counts[11][12]. Jin, Gawrychowski et al., Deng et al., have enhanced both the engineering and asymptotic characteristics of these methods. The studies illustrate that the use of dense-instance techniques or data-aware heuristics can significantly speed the run-time of the procedure on a number of real-world inputs[8][9][13].

Trimming — selective pruning of small coefficients — is ubiquitous in practical implementations that rely on polynomial representations. Without trimming, intermediate polynomial supports can explode, negating the benefits of fast convolution. Numerous engineering papers have studied heuristics and multiplicative trimming thresholds that yield dramatic memory/time reductions with limited accuracy

loss; Bateni et al. and later algorithm-engineering works show that careful choice of a trimming multiplier (and maximum-kept terms) is crucial for a favorable time–error trade-off [4]. Feng & Jin’s grouped DC framework formalizes trimming in the context of theoretical guarantees: by bounding group sizes and applying multiplicative trimming at each merge, they argue the intermediate growth can be controlled and sub-quadratic bounds can be approached in expectation under their model assumptions [1].

Parallelization choices introduce a second set of implementation-level tradeoffs. The grouped divide-and-conquer structure maps naturally to task parallelism: groups can be built, convolved, and trimmed independently, and then merged in a tree-like fashion. Distributed-memory (MPI) implementations therefore can accelerate the group-building and local merge phases by assigning groups to ranks, but communication and the root-level merging step can become bottlenecks if work is imbalanced or if trimmed polynomials are large. Conversely, GPU acceleration (via cuFFT) directly targets the heavy convolution operations: batched FFTs, multi-stream overlap, and kernel-level pointwise multiplications can be implemented efficiently, but the realized speedup depends strongly on FFT sizes (small FFTs are inefficient on GPUs), batch concurrency, and Host-to-Device/Device-to-Host transfer overheads. Thus offloading large FFTs to GPUs while keeping bookkeeping and fine-grained merges on CPUs are commonly proposed [14][15].

Despite strong theoretical and engineering progress, a gap remains: many recent sub-quadratic algorithmic papers focus on asymptotic guarantees or worst-case constructions and do not evaluate the practicalities of parallel and GPU implementations across heterogeneous platforms. In particular, how trimming policies interact with parallel merges, whether root-level merging can be parallelized or pipelined to avoid serial bottlenecks, and how to tune FFT sizes and batching for GPUs across a range of  $n$  and  $\epsilon$  values are open engineering questions. This work takes the Feng & Jin framework [1] as the starting point and contributes an empirical investigation that directly addresses these practical concerns: we measure scaling laws, trimming effects, and hardware tradeoffs, and identify bottlenecks (root merges, communication, and small-FFT inefficiencies) that suggest concrete directions for future research.

### 3 Methodology

The proposed system utilize and corroborate the optimized approach to previously defined 0-1 Knapsack problem implementation using Divide and Conquer (DC) and convolutional methods, alongside Fast Fourier Transform (FFT). The approach for our article is also explained in Paper [1], as proposed theoretical inference that accomplished this sub-quadratic time  $O(n^{1.5})$  time complexity reduction, via grouped dynamic programming and FFT based merging.

Thus, our workflow entails random item generation, polynomial groups, recursive convolution merging, ultimately presenting results in various configurations and we implemented our theoretical model in 3 versions:

3.1 Sequential CPU Version

3.2.1 Parallel MPI version

3.2.2 GPU accelerated CUDA version with cuFFT

### 3.1 Sequential Implementation

This version forms the foundation for further verification and correlation between results published here as it relates solely to instances of knapsack produced under specified parameters: a total of 1000 items ( $n = 1000$ ) and a knapsack capacity of 100 ( $C = 100$ ) with a maximum weight limit of 200 ( $w_{max} = 200$ ) for each item, and an error approximation factor ( $\mathcal{E} = 0.05$ ). The implementation of the FFTW3 Library was utilized within the context of this work to perform Fourier Transformation of its FFTW3 based data.

Initially, in order to generate random weights and profits for all instances of knapsack, a deterministic seed was provided for all random values. Each item pair  $(w_i, p_i)$  satisfied the conditions  $1 \leq w_i \leq w_{max}$  and  $1 \leq p_i \leq 15$ . To aid in reducing computation time of our implementation, a scale factor was used to convert from item weight (gamma) to scaled item weight as illustrated by the following equation:  $\gamma = \mathcal{E} \times (w_{max}/n)$ . After that, items were sorted by their scaled weights and divided into  $\sqrt{n}$  partitions.

Finally, after generating partitioned sets of items, convolutional merging of partitioned sets of items occurred via FFT, specifically FFTW3 Real to Complex and Complex to Real transforms, thus reducing the computational requirement of our implementation from  $O(n^2)$  to  $O(n \log n)$ . The polynomial representation for each item ( $P_i$ ) can be defined as  $P_i(x) = 1 + x^{w_i}$ . Partitioned sets of items were combined recursively by means of a divide and conquer (DC) FFT strategy to combine all combinations which could fit under the weight limit  $C$ .

To ensure that polynomial growth is controlled and at the same time to ensure that a sparse representation is preserved, coefficients that were lower than a pre-defined truncation threshold of  $\delta = \mathcal{E}/4$  were also eliminated from further consideration. This process of selective pruning is an effective way to reduce the amount of memory consumed while maximally preserving the approximation accuracy. A composite merged polynomial containing all the possible combinations of items that meet the capacity constraint has been created, which allows for an implemented sequential process to provide a report of the build time, merge time, and the overall run time. The theoretical time complexity of this method is:  $O(n^{1.5} \text{poly} \log(1/\mathcal{E}))$ .

---

#### Algorithm I: Sequential implementation

**Input:** Items  $(w_i, p_i)$ , capacity  $C$ ,  $\mathcal{E}$

**Output:** Approximate feasible combinations within  $C$

Step 1: Initialize parameters:  $n, w_{max}, \mathcal{E}, C$

Step 2: Compute scaling factor:  $\gamma = \mathcal{E} \times (w_{max}/n)$

Step 3: Scale all item weights:  $w'_i = \text{floor}(w_i / \gamma)$ .

Step 4: Partition items into  $G = \text{ceil}(\sqrt{n})$  groups.

Step 5: For each group: Construct polynomial  $P_{g(x)} = \pi(1 + x^{w_i})$ .

Step 6: Merge groups recursively using FFT convolution  $P_{merge} \leftarrow$

$\text{FFT\_Convolve}(P_{g_1}, P_{g_2})$ .

Step 7: Apply trimming: discard *coefficients*  $< \delta = \mathcal{E} - 4$

Step 8: Return the final polynomial  $P_{f(x)}$  representing all feasible subsets

---

**Fig. 1.** Sequential implementation of the DC knapsack.

### 3.2.1 Parallel Implementation using MPI

The design of creating a parallel implementation with MPI was to improve the speed of a sequential workflow through sharing the workload of a group computation among many different ranks. The experiment was performed under an Ubuntu 20.04 system using MPI 4.0.3, GCC 9.4.0, and the FFTW3 libraries. The general outline for the experimental setup was broken down into four major areas of interest, including Data Partitioning, Computation Distribution, Synchronization & Gathering, and Timing Measurement.

For Data Partitioning, each group of size  $\sqrt{n}$  created by the sequential method was divided evenly among all of the MPI ranks. Each process then performed its own independent local merging of groups using the same FFT-based convolution process that was used for the baseline method. During Computation Distribution, each rank then created its own local sparse polynomial and had to trim the polynomial to keep the sparsity of it. The FFT merge counters and enumerations are maintained separately for each processor to make sure the workload and local results are balanced and accurate.

In the Synchronization and Gathering phase, the root process received intermediate results that had been sent from all of the ranks, which it then combined into a comprehensive representation of the final global polynomial formulation. The timing for measuring was made using the MPI Wtime() method to examine the time of build and merging phases between ranks. The total wall-clock duration for all ranks was substantially lower than that of the Sequential Baseline, confirming the distributed approach to be scalable.

Rank 0, in the MPI implementation, serves as the master process that sets up the environment, loads the signature database, and generates the complete list of files to analyze. After completing the initialization phase, Rank 0 broadcasts both the signature and file lists to all of the participating processes. Each of the processes was allocated its share of the total workload using a basic static partitioning technique.

The largest benefit of implementing MPI in parallel processing is that it can considerably decrease the time taken for each execution as multiple processes perform computations simultaneously. The communication cost and subsequent merging at the root process do impact performance where the performance for group-based convolution as opposed to single-threaded CPU processing has been considerably increased using this implementation compared to a single CPU execution.

Timing Measurement is given by:

$$T_{total} = \max(T_{build}^{(rank\ i)}) + T_{merge}^{(root)}$$

Speedup Evaluation:

$$Speedup = T_{seq} - T_{mpi}$$


---

#### **Algorithm II: MPI Implementation**

**Input:** Items  $(w_i, p_i)$ ,  $\mathcal{E}$ , ranks =  $p$

**Output:** Approximate knapsack polynomial  $P_{f(x)}$ .

- Step 1:* Broadcast  $n$ ,  $\epsilon$ , and group data to all ranks.
- Step 2:* Each rank builds local groups and performs FFT merges.
- Step 3:* Apply trimming locally after each merge.
- Step 4:* Record  $\text{local\_time} = \text{MPI\_Wtime}()$ .
- Step 5:* Root process gathers all results using  $\text{MPI\_Gather}()$ .
- Step 6:* Root merges partial groups via FFT convolution.
- Step 7:* Compute total execution time and speedup.
- Step 8:* Output final merged polynomial  $P_{f(x)}$ .

---

**Fig. 2.** MPI implementation of DC-FFT Knapsack.

Time complexity for this implementation:  $O(n^{1.5}/p)$

### 3.2.2 Parallel Implementation using CUDA

Next, to further accelerate the FFT-based merging process, a GPU implementation was developed using CUDA 12.4 and the cuFFT library and tested on a Tesla T4 GPU using Google Colab. There are several compute stages that contribute to the full sequence of computations within the workflow: Data Transfer, Forward FFT Transformation, Pointwise Multiplication, Inverse FFT Transformation and Batched Multi-Stream Processing, with a summary of the overall depth of computation as follows:

$$C = \text{IFFT}(\text{FFT}(A) \times \text{FFT}(B))$$

In the phase of transferring data from the host to the device, the data arrays that represent the item's weights and profits are copied from the host (CPU) memory into device (GPU) memory. Each input sequence is subsequently padded to the nearest power-of-two length in order to allow for optimal execution by cuFFT and to eliminate wrap-around problems in the circular convolution. The latter issue results from the fact that the input cannot be divided into equal portions of a specification defined by cuFFT.

When padded input arrays are transformed at the Forward FFT Transformation with the use of the function `cufftExecD2Z()`, the input array will have been transformed using a real to complex double precision Fast Fourier Transform (FFT). This transformation represents a time-domain sequence in the frequency domain so convolution can be calculated as a two-dimensional array of the two-dimensional frequency domain element wise product.

The Pointwise Multiplication process uses a Custom CUDA Kernel to perform elementwise Multiplication of the transformed arrays. During this process, the kernel Applies Normalization, thereby maintaining amplitude throughout. The Mathematical Representation of the Kernel is a description of how each thread returns the product of two complex coefficients along with the intermediate sums. The Designed Kernel takes advantage of High Thread Parallelism of the GPU and the Shared Memory of the GPU to Reduce Latency and Computational Throughput.

$$C_k = (A_r B_r - A_i B_i) + j(A_r B_i + A_i B_r)$$

Next, The merged polynomial that was constructed at the end of the process through the Inverse Fast Fourier Transform transformation `cufftExecZ2D()` (Complex to Real FFT) is generated. Following this, each coefficient in the resulting array is scaled using a constant value of  $1/n$  to return the convolution to its original value or normalised form.

---

**Algorithm III: CUDA Implementation**

**Input:** Double precision arrays:  $A, B$

**Output:** Convolved array:  $C = A \otimes B$

- Step 1:* Allocate device memory and copy  $A, B$  to GPU
  - Step 2:* Execute `cufftExecD2Z()`  $\rightarrow A_f, B_f$
  - Step 3:* Launch kernel:  $A_f[i] = A_f[i] \times B_f[i]/n$ .
  - Step 4:* Execute `cufftExecZ2D()`  $\rightarrow C$
  - Step 5:* Copy  $C$  back to host
  - Step 6:* Repeat for multiple streams to enable overlapping
  - Step 7:* Return  $C$  and record GPU timing metrics.
- 

**Fig. 3.** CUDA implementation of FFT-based Convolution Knapsack.

Time complexity for this implementation:  $O(n^{1.5}/G_{gpu})$

## 4 Results and Discussion

Three different versions of the FFT Approximation Framework were tested for the 0-1 Knapsack Problem in an experimental setting: Sequential Implementation (CPU), MPI based Parallel Implementation, and CUDA based GPU Implementation. The test environment was configured with varying degrees of input size  $n = 1000, 5000, 10000$ , a knapsack capacity  $C = 200$ , and an approximation parameter  $\varepsilon = 0.05$ .

The MPI (Message Passing Interface) Implementation was created to maximize the grouped dynamic programming workflow through the parallelisation of the  $n$  groups by distributing each group across several processes. The local FFT convolution and trimming was done independently on each MPI rank and then there was a merging phase where everything was brought back to the root process.

Figure 1 shows that while the execution time decreases significantly with the addition of two more ranks on the MPI side, the rate of improvement is not nearly as rapid after this initial increase. At 8 ranks, the rate of improvement has slowed down noticeably and it is likely due to multiple factors such as the additional time taken for inter-process comms, unequal loads between ranks, and delays from the coordination of the data gathered and merged.

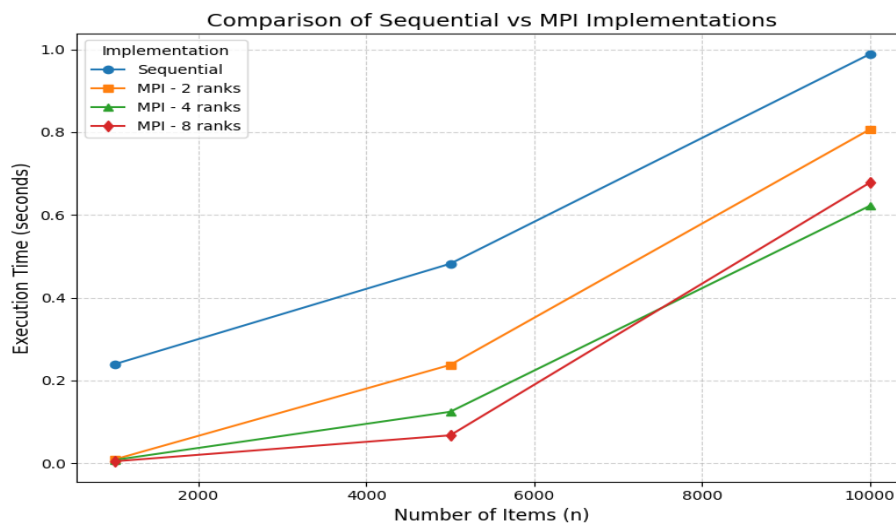
Communication costs tend to exceed computational benefits for smaller datasets ( $n \leq 1000$ ), but for larger instances ( $n = 10000$ ), Although parallel FFT knapsack



problem algorithms provide improved performance compared to sequential algorithms, diminishing returns are observed when using greater than the minimum number of processes. This observation supports Amdahl's Law: the sequential section of an algorithm (most often the root merge operation) determines the maximum attainable speedup of an algorithm.

| Input data size<br>( $n$ ) | Sequential<br>execution time<br>(seconds) | Number of<br>Processors | MPI Execution<br>time (seconds) | Speedup(x) |
|----------------------------|-------------------------------------------|-------------------------|---------------------------------|------------|
| 1000                       | 0.239400                                  | 2                       | 0.009324                        | 25.67      |
|                            |                                           | 4                       | 0.008018                        | 29.85      |
|                            |                                           | 8                       | 0.004824                        | 49.62      |
| 5000                       | 0.482018                                  | 2                       | 0.237604                        | 2.02       |
|                            |                                           | 4                       | 0.124380                        | 3.87       |
|                            |                                           | 8                       | 0.067554                        | 7.14       |
| 10000                      | 2.000000                                  | 2                       | 0.067554                        | 2.04       |
|                            |                                           | 4                       | 0.631730                        | 3.21       |
|                            |                                           | 8                       | 0.607171                        | 3.29       |

**Table 1.** Execution Time and Speedup of MPI over Sequential



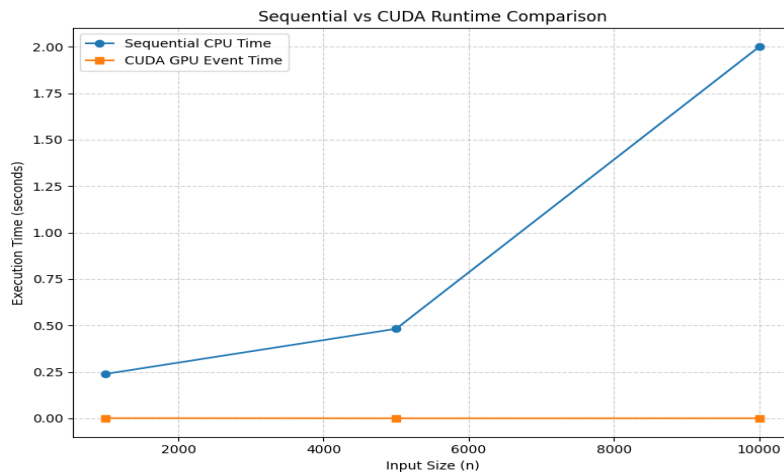
**Fig 4.** Execution Time plot of Sequential vs MPI

The development of a CUDA kernel allows the programmer to use the parallel processing capabilities of GPU to speed up the FFT based Convolution and Merging operations. The CUDA implementation was developed using the cuFFT library, which is part of the CUDA Toolkit, and it ran on an NVIDIA Tesla T4 GPU. The run time of the CUDA kernel was constant for each of these cases regardless of how large or small the input lists were which illustrates that GPU parallelism is advantageous in terms of both Scalability and Throughput.

In addition, because large amounts of time spent on runtime differences was attributed to the costs associated with data transfer and initializing kernels, not kernel execution times, it was found (by GPU event timing) that only the actual computation times of kernels were considered for total computation performance and thus provide an accurate estimate of the raw compute efficacy of the architecture.

| $n$   | Sequential Time(sec) | CUDA Time(sec) | Speedup(x) |
|-------|----------------------|----------------|------------|
| 1000  | 0.239400             | 0.001569       | 152        |
| 5000  | 0.482018             | 0.000696       | 693        |
| 10000 | 2.000000             | 0.000915       | 2168       |

**Table 2.** Speedup comparison table of CUDA vs Sequential



**Fig 5.** CUDA vs Sequential runtime speedup graph

In general, a consistent trend can be observed with the sequential implementation showing a growth rate like  $O(n^{1.5})$ . The MPI implementation allows for a reasonable amount of speed up for smaller and medium size workloads, there is a decline in performance because of communication bottleneck issues. The CUDA implementation on the other hand delivers excellent performance gains from its ability to perform many computations in parallel through the use of fine grain (small) parallelism with optimized cuFFT capabilities.

Both MPI & CUDA accelerate the performance of the grouped dynamic programming based on dynamic programming compared to the sequential baseline, with the highest level of performance increase when using CUDA. Thus, the results provide empirical evidence that Feng & Jin's theoretical sub-quadratic design is optimally realisable on today's leading-edge computing architectures, especially on systems with the ability to run FFT computations in parallel via a GPU.

## 5 Conclusion

The MPI implementation provides a moderate speedup for small to medium workloads, but for larger ranks it has reduced efficiency from overhead associated with sending and receiving messages.

In particular, the CUDA implementation achieves very high levels of performance by utilizing GPU parallelism in conjunction with cuFFT-based batched transforms to accelerate convolution and polynomial merging (i.e. combining coefficients). The execution time for the kernels remained virtually stable at every level of the problem, producing speedup ratios above 2000x for the largest problems. The advantage of this type of parallelism is that it can execute FFT-dominated workloads with virtually no increase in the amount of time taken to complete them.

In conclusion, the theoretical framework for counting knapsack solutions using a sub-quadratic algorithm put forth by Feng and Jin has been demonstrated to be feasible in the modern approaches to implementing the algorithms described above on modern heterogeneous architectures. Although performance improvements have been achieved through the use of MPI for distributed computing, these improvements will not be as substantial in developing the efficiencies offered by GPU accelerators (i.e. speedups).

Further development may come from the implementation of hybrid CPU and GPU systems as well as developing better methods of balancing workloads among cores (e.g. dynamic workload balancing) between different MPI ranks and therefore using new ways to balance workloads based on how an application is executed, such as altering the parameters ( $\mathcal{E}$  and  $\gamma$ ) in the trim algorithm when the application is executed. The use of multiple GPUs and multiple node clusters also extends the ability to solve larger problems by providing larger amounts of hardware resources.

## References

1. Feng, Weiming, and Ce Jin.: Approximately Counting Knapsack Solutions in Subquadratic Time. In: Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 1094-1135, Society for Industrial and Applied Mathematics (2025).
2. Bellman, R.I.C.H.A.R.D.: Dynamic programming, princeton univ. press, New Jersey, 39 (1957).
3. Kellerer, Hans, Ulrich Pferschy, and David Pisinger.: Multidimensional knapsack problems. In: Knapsack problems, pp. 235-283. Berlin, Heidelberg: Springer Berlin Heidelberg (2004).
4. Bateni, Hossein Md, Hajiaghayi Md, Seddighin S, and Cliff Stein.: Fast algorithms for knapsack via convolution and prediction. In: Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, pp. 1269-1282 (2018).
5. Bringmann, Karl, and Alejandro Cassis.: Faster 0-1-knapsack via near-convex min-plus-convolution. arXiv preprint arXiv:2305.01593 (2023).
6. Bringmann, Karl, and Alejandro Cassis.: Faster knapsack algorithms via bounded monotone min-plus-convolution. arXiv preprint arXiv:2205.08493 (2022).
7. Bringmann, Karl, Anita Dürr, and Adam Polak.: Even faster knapsack via rectangular monotone min-plus convolution and balancing. arXiv preprint arXiv:2404.05681 (2024).
8. Gawrychowski, Paweł, Liran Markin, and Oren Weimann.: A faster FPTAS for# knapsack. arXiv preprint arXiv:1802.05791 (2018).
9. Jin, Ce.: An improved FPTAS for 0-1 knapsack. arXiv preprint arXiv:1904.09562 (2019).
10. Bringmann, Karl, and Vasileios Nakos.: A fine-grained perspective on approximating subset sum and partition. In: Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 1797-1815. Society for Industrial and Applied Mathematics (2021).
11. Dyer, Martin.: Approximate counting by dynamic programming. In: Proceedings of the thirty-fifth annual ACM symposium on Theory of computing, pp. 693-699 (2003).
12. Gopalan, Parikshit, Adam Klivans, Raghu Meka, Daniel Štefankovic, Santosh Vempala, and Eric Vigoda.: An FPTAS for# knapsack and related counting problems. In: 2011 IEEE 52nd Annual Symposium on Foundations of Computer Science, pp. 817-826. IEEE (2011).
13. Deng, Mingyang, Ce Jin, and Xiao Mao.: Approximating knapsack and partition via dense subset sums. In: Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 2961-2979. Society for Industrial and Applied Mathematics (2023).
14. OpenMPI Team: OpenMPI 4.0.7 Documentation, <https://www.open-mpi.org/doc/>, last accessed 2025/11/10.
15. NVIDIA: cuFFT Developer Guide, <https://docs.nvidia.com/cuda/cufft/>, NVIDIA Corporation, last accessed 2025/11/10.